

Kreacijski dizajn paterni

U okviru ovog dokumenta opisano je 5 kreacijskih paterna, kao i njihove potencijalne primjene u sistemu ETFPay. Kao primjer za implementaciju na dijagramu klasa uzeti su Builder i Prototype.

1. Singleton

Singleton osigurava da klasa ima samo jednu instancu u cijelom sistemu i osigura globalni pristup kreiranoj instanci klase. Također se koristi za odgođeno (lazy) instanciranje objekata. Kako bi se izbjegle greške s različitim kursovima valuta u istom trenutku, Singleton osigurava da u memoriji postoji samo jedan objekat klase KursnaLista. To se postiže privatnim konstruktorom koji zabranjuje direktno kreiranje novih objekata i statičkom metodom getInstance(), koja pri prvom pozivu kreira taj jedinstveni objekat, a pri svakom sljedećem pozivu vraća tu istu instancu. Na ovaj način, svi dijelovi sistema dijele i koriste isti izvor podataka.

2. Prototype

Služi za kreiranje novih objekata kloniranjem postojećih, čime se izbjegava ponovni unos podataka i ručno kreiranje objekta iznova. Ovaj patern je idealan kada je proces instanciranja objekta skup u smislu performansi ili kada objekti zahtijevaju kompleksno inicijalno konfigurisanje.

Unutar klase Predlozak kreira se metoda kloniraj() koja pravi kopiju starog šablona. Umjesto da klijent svaki put unosi podatke ispočetka i sistem kreira novi objekat putem konstruktora. Na tom novom objektu aplikacija samo promijeni specifične detalje (poput iznosa), dok originalni šablon ostaje netaknut.

3. Factory Method

Factory Method patern omogućava kreiranje objekata tako što odluku o izboru konkretne klase za instanciranje prebacuje na podklase.

U našem sistemu, umjesto direktnog pozivanja <<Create>> NovaOsoba(), može se definisati OsobaFactory sa metodom kreirajOsobu(Uloga). Zavisno od prosljeđene uloge (klijent, uposlenik, administrator), Factory Method vraća odgovarajuću instancu.

4. Abstract Factory

Pružanje interfejsa za kreiranje familija povezanih objekata bez specificiranja njihovih konkretnih klasa.

Kada bi sistem nudio različite pakete za klijente, npr. "Studentski Paket" i "Poslovni Paket". StudentskiPaketFactory bi kreirala objekte StudentskiRacun, osnovni Predlozak sa ograničenim brojem transakcija i standardnu Karticu. PoslovniPaketFactory bi kreirala objekte PoslovniRacun, napredni Predlozak bez ograničenja i zlatnu Karticu. Ovaj patern garantuje da će se objekti iz iste familije uvijek kreirati zajedno i da klijent neće greškom dobiti studentski račun sa poslovnom karticom.

5. Builder

Builder odvaja specifikacije kompleksnih objekata od njihove stvarne konstrukcije. Koristi se kada objekat sadrži veliki broj atributa.

Kako transakcija zahtijeva pošiljaoca, primaoca, iznos, datum i status aktivacije, uvodimo pomoćnu klasu TransakcijaBuilder. Proces kreiranja se odvija korak po korak pozivanjem ulančanih metoda poput `.setPosiljalac()`, `.setPrimalac()`, `.setIznos()`. Unutar samog buildera, prije nego što se pozove finalna metoda `.build()` koja zapravo kreira i vraća gotov objekat Transakcija, vrše se ključne validacije (npr. provjera stanja na računu pošiljaoca, validnost IBAN-a).